

Caracterização da Complexidade Estrutural em Projetos Spring Boot

Daniel Estevam¹, Guilherme Augusto¹, Matheus Hoske¹,
Otávio Júlio¹, Samuel Almeida¹, Rafael Marques¹

¹Engenharia de Software – Pontifícia Universidade Católica de Minas Gerais (PUC-MG)
Belo Horizonte – MG – Brazil

daniel.estevam@sga.pucminas.br

g2002souzajardim@gmail.com

matheus.hoske@sga.pucminas.br

otaviojulio.mendes@gmail.com

samuel.almeida@sga.pucminas.br

rafael.radieddine@sga.pucminas.br

Abstract. *This paper presents an exploratory analysis of the structural complexity of Java projects developed with the Spring Boot framework. The study focuses on object-oriented metrics related to coupling, class and method complexity, and inheritance depth, extracted through the CK tool and consolidated using an ETL process in Power BI. Public GitHub repositories were analyzed to identify patterns in metrics such as CBO, RFC, WMC, DIT, and NOC, enabling the characterization of how modern Spring Boot applications organize their internal structure. The results highlight trends related to coupling levels, method verbosity, and the predominance of shallow inheritance hierarchies, contributing to a broader understanding of architectural practices and structural qualities in contemporary Java systems.*

Resumo. *Este artigo apresenta uma análise exploratória da complexidade estrutural de projetos Java desenvolvidos com o framework Spring Boot. O estudo concentra-se em métricas orientadas a objetos relacionadas a acoplamento, complexidade de classes e métodos e profundidade de herança, extraídas por meio da ferramenta CK e consolidadas em um processo de ETL no Power BI. Repositórios públicos do GitHub foram analisados para identificar padrões em métricas como CBO, RFC, WMC, DIT e NOC, permitindo caracterizar como aplicações modernas em Spring Boot organizam sua estrutura interna. Os resultados evidenciam tendências relacionadas a níveis moderados de acoplamento, influência da verbosidade na complexidade e predominância de hierarquias rasas de herança, contribuindo para a compreensão de práticas arquiteturais e qualidades estruturais em sistemas Java contemporâneos.*

1. Introdução

A qualidade interna de software está diretamente associada à forma como módulos, classes e métodos são estruturados, o que influencia tanto o esforço de manutenção quanto a capacidade de evolução ao longo do ciclo de vida. Nesse contexto, métricas internas têm sido amplamente empregadas para quantificar propriedades estruturais como tamanho, acoplamento, coesão e herança, apoiando avaliações de design e decisões relacionadas à organização do código [Singh and Verma 2012, Gupta et al. 2015]. Entre essas métricas, o acoplamento entre classes (*Coupling Between Objects*, CBO) destaca-se por refletir o grau de dependência entre componentes, sendo frequentemente utilizado para caracterizar complexidade estrutural em sistemas orientados a objetos [Child et al. 2019].

Aspectos arquiteturais contemporâneos, como o uso extensivo de injeção de dependências (*Dependency Injection*, DI) em ecossistemas Java, também impactam o modo como as dependências são declaradas e gerenciadas no código. Em *frameworks* como o *Spring Boot*, a DI promove desacoplamento explícito de componentes, podendo alterar a leitura tradicional de métricas relacionadas a dependências diretas [Sun et al. 2022]. Da mesma forma, a organização das classes em hierarquias mais rasas ou mais profundas pode influenciar a compreensibilidade e o reuso, reforçando a importância de métricas como DIT e NOC para compreender a estrutura interna de projetos Java modernos.

Embora existam estudos que investigam métricas de orientação a objetos em diferentes contextos, ainda é limitada a caracterização empírica de projetos desenvolvidos com *Spring Boot* em larga escala, especialmente no que se refere à combinação entre acoplamento, complexidade e herança. Este cenário destaca a relevância de análises que utilizem métricas consolidadas, aplicadas a repositórios públicos, para identificar padrões estruturais e práticas predominantes em aplicações contemporâneas baseadas nesse *framework*.

Diante disso, surge o objetivo principal deste trabalho: **analisar projetos Java open-source desenvolvidos com *Spring Boot* com o propósito de caracterizar sua complexidade estrutural por meio de métricas orientadas a objetos extraídas de repositórios públicos do GitHub**. Para operacionalizar esse objetivo, investigam-se as seguintes questões de pesquisa:

- **Q1** — Qual é o volume de acoplamento entre classes nos projetos *Spring Boot*.
- **Q2** — A complexidade estrutural dos projetos é influenciada pela densidade lógica dos métodos, representada pelo tamanho médio das funções.
- **Q3** — Como a hierarquia de herança está estruturada nos projetos *Spring Boot*.

Este artigo está organizado da seguinte forma, a Seção 2 apresenta os trabalhos relacionados e o posicionamento do estudo, a Seção 3 descreve o desenho metodológico e o processo de coleta e análise das métricas, a Seção 4 apresenta os principais resultados, a Seção 5 discute suas implicações, a Seção 6 detalha as ameaças à validade e, por fim, a Seção 7 apresenta as conclusões e sugestões para trabalhos futuros.

2. Trabalhos Relacionados

A literatura sobre métricas orientadas a objetos oferece bases importantes para a compreensão de acoplamento, complexidade e herança em sistemas Java, incluindo aplicações desenvolvidas com *frameworks* modernos. Sun et al. (2022) investigam como a injeção

de dependências (*Dependency Injection*, DI) influencia a interpretação tradicional de métricas de acoplamento e propõem extensões capazes de capturar dependências mediadas por contêineres. Embora o presente estudo utilize apenas métricas clássicas, os achados de Sun et al. reforçam a necessidade de considerar o papel arquitetural da DI ao analisar projetos *Spring Boot*.

Child, Rosner e Counsell (2019) mostram que diferentes ferramentas e estudos utilizam variantes distintas do CBO, o que pode gerar resultados inconsistentes. Essa evidência destaca a importância de documentar claramente a definição adotada e manter coerência metodológica, diretrizes seguidas neste trabalho por meio do uso explícito da ferramenta CK.

Tahir et al. (2021) investigam a influência do tamanho de classes e métodos sobre métricas estruturais e ressaltam que medidas de complexidade e acoplamento são sensíveis à granularidade do código. Essa observação fundamenta a análise conjunta entre WMC, tamanho médio dos métodos e métricas de acoplamento realizada no presente estudo.

Anwer et al. (2017) analisam a decomposição do acoplamento em dimensões referentes e eferentes e mostram que diferentes tipos de dependência refletem papéis distintos dentro da arquitetura. Embora o estudo atual não empregue Ca e Ce, esses trabalhos reforçam a relevância de investigar acoplamento sob múltiplas perspectivas e motivam o foco em métricas relacionadas a dependências diretas em projetos *Spring Boot*.

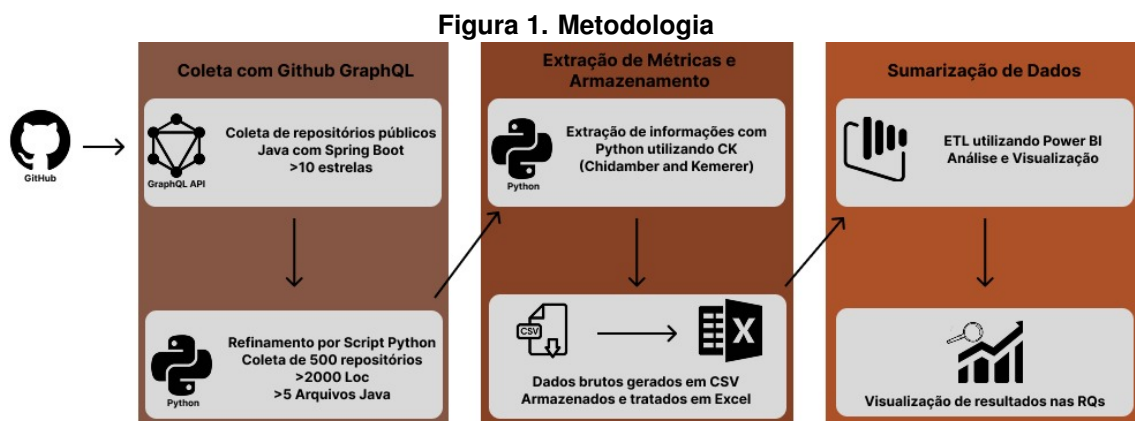
Por fim, Singh e Verma (2012) e Gupta et al. (2015) demonstram a utilidade prática de métricas internas para compreender características estruturais de software e apoiar avaliações de qualidade em ambientes *open source*. Esses trabalhos justificam a escolha do conjunto de métricas utilizado, bem como a análise descritiva aplicada aos repositórios públicos avaliados.

Em síntese, os estudos revisados fornecem fundamentos conceituais sobre variações de métricas, influência de fatores estruturais, impacto da DI em acoplamento e interpretação adequada de complexidade, oferecendo suporte direto ao desenho analítico utilizado neste trabalho.

3. Metodologia

3.1. Visão geral e fluxo do estudo

O estudo segue o modelo GQM, cujo objetivo é caracterizar a complexidade estrutural de projetos Spring Boot a partir de métricas orientadas a objetos extraídas com a ferramenta CK. As três questões de pesquisa investigam o acoplamento entre classes, a relação entre verbosidade e complexidade estrutural e a estrutura de herança. O processo metodológico é composto por três etapas principais, conforme ilustrado na Figura 1: coleta dos repositórios no GitHub, extração automática de métricas de código com a ferramenta CK e consolidação das métricas por meio de ETL no Power BI para análise e visualização dos resultados.



Fonte: Elaborada pelo autor

3.2. Etapa 1 – Seleção dos repositórios

A seleção da amostra foi realizada por meio da GitHub GraphQL API, utilizando um script em Python com paginação automática. Foram coletados 500 repositórios públicos escritos majoritariamente em Java e que utilizam Spring Boot, identificados pela presença de dependências spring boot starter nos arquivos pom.xml ou build.gradle. Para garantir relevância mínima, foram aplicados os seguintes critérios de inclusão: linguagem primária igual a Java, uso explícito de Spring Boot, repositórios não forkados, pelo menos 10 estrelas e tamanho mínimo aproximado de 2000 linhas de código efetivas. Os metadados coletados incluem identificador do repositório, URL, número de estrelas, data da última atualização e estimativa de tamanho do projeto.

3.3. Etapa 2 – Extração de métricas de código

Os repositórios elegíveis foram clonados automaticamente e processados pela ferramenta CK, que calcula métricas orientadas a objetos amplamente utilizadas na literatura. As métricas extraídas para cada classe Java foram: CBO, RFC, DIT, NOC, WMC e LOC. Os arquivos CSV gerados pela ferramenta foram armazenados para posterior integração e análise. Em caso de falhas no processo de clonagem ou execução da ferramenta, o repositório era substituído por outro da lista até que o alvo amostral fosse atingido.

3.4. Etapa 3 – Consolidação e integração dos dados

Os dados produzidos pela CK foram integrados em um processo de ETL realizado no Power BI. Nesta etapa foram tratados dados inconsistentes, removidas duplicidades e calculadas métricas derivadas como médias agregadas por projeto. Essas métricas incluem *avg cbo*, que representa o nível médio de acoplamento entre classes ao indicar quantas outras classes cada classe utiliza diretamente, *avg rfc*, que expressa a quantidade média de métodos que podem ser acionados em resposta a uma mensagem enviada a uma classe, refletindo a amplitude de comportamentos disponíveis, *avg dit*, que descreve a profundidade média da árvore de herança ao medir quantos níveis separam uma classe da raiz da hierarquia, *avg noc*, que corresponde ao número médio de subclasses diretas por classe e indica o grau de ramificação da herança no projeto, *avg wmc*, que reflete a complexidade lógica agregada das classes por meio do somatório das complexidades ciclomáticas de

seus métodos, e *avg loc method*, que caracteriza a verbosidade do código ao representar o tamanho médio dos métodos calculado pela razão entre o total de linhas de código de métodos e a quantidade de métodos presentes no projeto. O processo resultou em um conjunto consolidado utilizado para responder às três questões de pesquisa.

4. Resultados

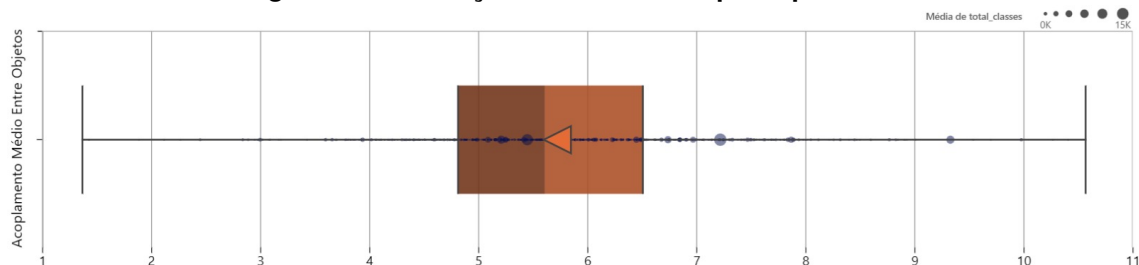
4.1. Q1, Qual é o volume de acoplamento entre classes nos projetos Spring Boot

A amostra analisada apresenta projetos Spring Boot com tamanhos variados em termos de classes e funcionalidades, o que se reflete em uma distribuição ampla das métricas de acoplamento. A Figura 2 mostra a distribuição do CBO médio por repositório. Os valores variaram entre 1,37 e 10,57, com mediana de 5,61 e média global de 5,73, indicando que, em um cenário típico, cada classe depende de cinco ou seis outras classes em média.

A Figura 3 apresenta os valores médios de RFC por projeto, que variaram entre 1,05 e 47,24, com mediana de 8,19. Isso significa que, para a maior parte dos sistemas, uma classe pode acionar entre seis e onze métodos em resposta a uma mensagem recebida.

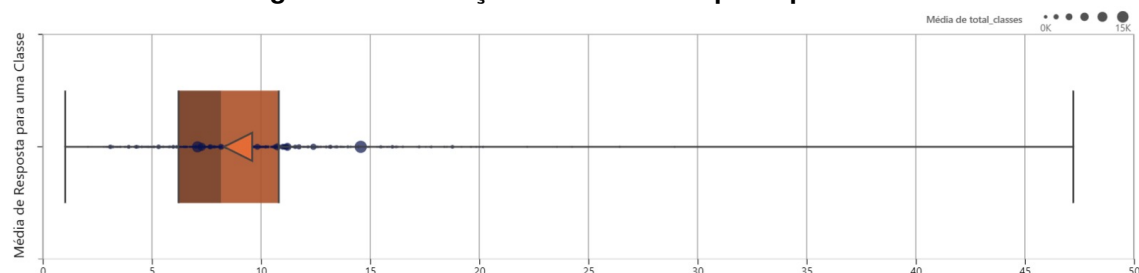
De forma geral, CBO e RFC sugerem um padrão de acoplamento moderado na maior parte dos projetos Spring Boot analisados. Esse comportamento é coerente com a arquitetura típica do framework, que organiza responsabilidades em camadas e utiliza injeção de dependências para reduzir acoplamentos diretos. Ainda assim, alguns projetos apresentam valores significativamente mais altos, revelando classes que concentram dependências e podem demandar atenção em processos de refatoração.

Figura 2. Distribuição do CBO médio por repositório



Fonte: Elaborada pelo autor

Figura 3. Distribuição do RFC médio por repositório



Fonte: Elaborada pelo autor

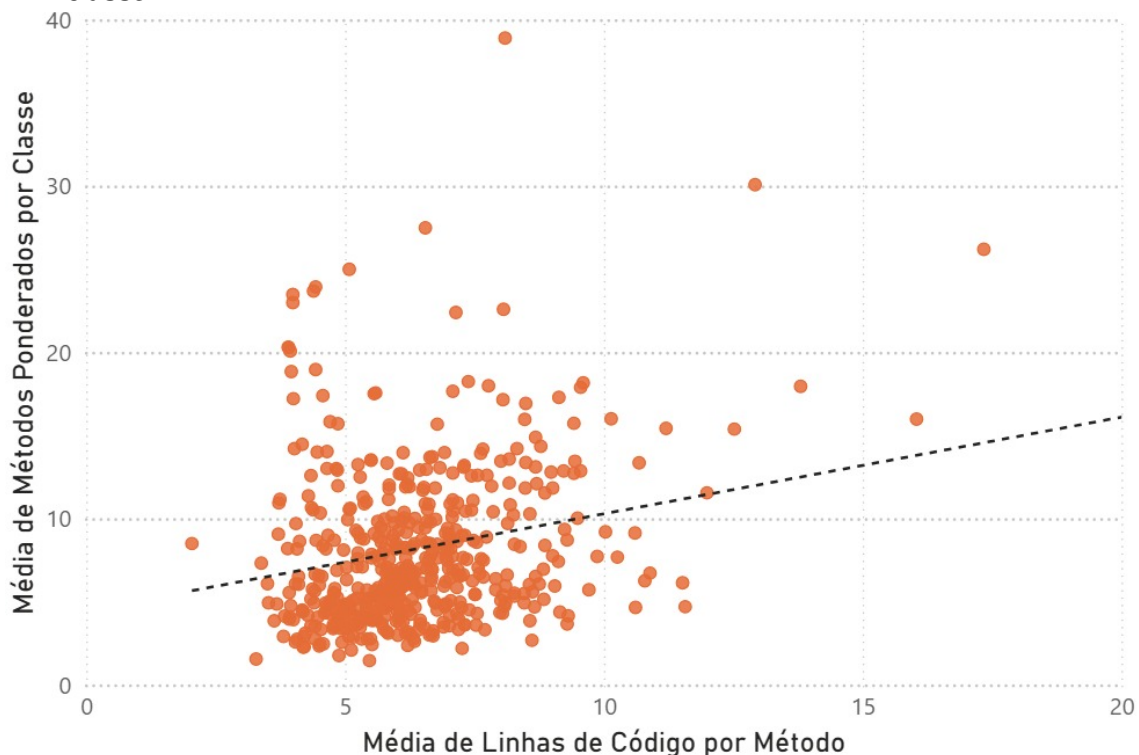
4.2. Q2, A complexidade estrutural dos projetos é impulsionada pela densidade lógica

A segunda questão de pesquisa investiga se a verbosidade dos métodos está associada ao aumento da complexidade estrutural das classes. Para isso foram analisadas as métricas *avg loc method* e *avg wmc*. A Figura 4 apresenta o gráfico de dispersão entre essas duas variáveis.

O tamanho médio dos métodos variou entre 2,04 e 26,81 linhas, com média de 6,50, enquanto o *avg wmc* variou entre 1,47 e 69,90, com média de 8,26. Observa-se uma tendência geral de crescimento conjunto: projetos com métodos mais longos tendem a apresentar maior complexidade ciclomática agregada por classe.

Esse resultado é consistente com a hipótese H2.1, segundo a qual a densidade lógica dos métodos influencia diretamente o esforço cognitivo necessário para compreender a classe. Embora exista dispersão, indicando variações de estilo e de organização interna entre os projetos, o padrão global sugere que métodos longos contribuem para o aumento da complexidade estrutural.

Figura 4. Relação entre tamanho médio dos métodos e complexidade média por classe



Fonte: Elaborada pelo autor

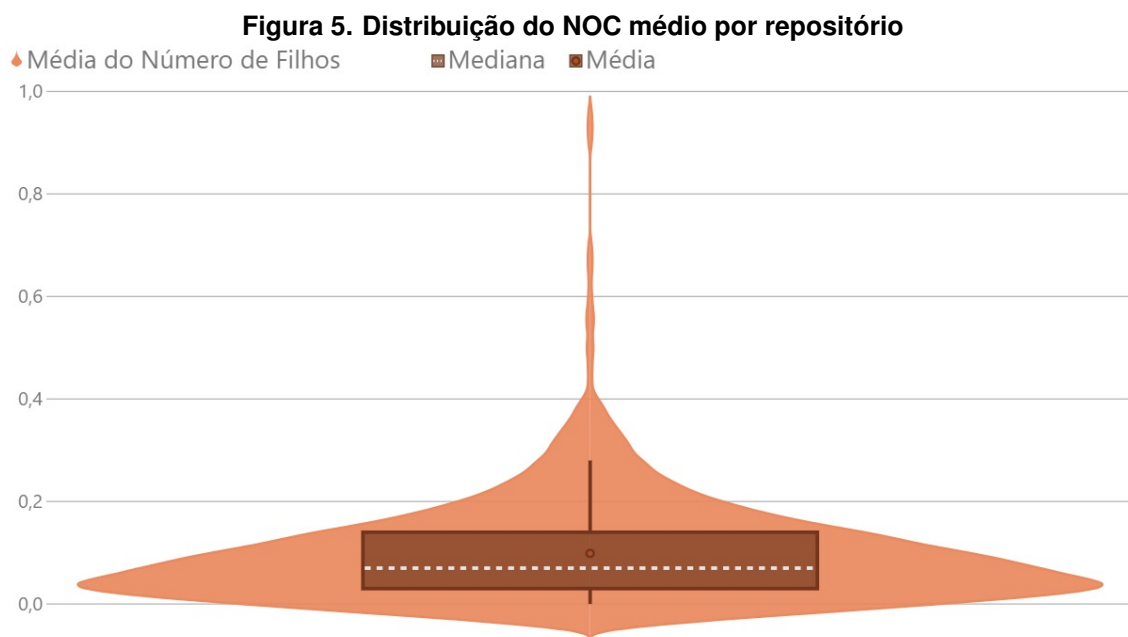
4.3. Q3, Como a hierarquia de herança está estruturada nos projetos Spring Boot

A terceira questão de pesquisa avalia a estrutura de herança dos projetos analisados por meio das métricas NOC e DIT. A Figura 5 apresenta a distribuição do NOC médio por

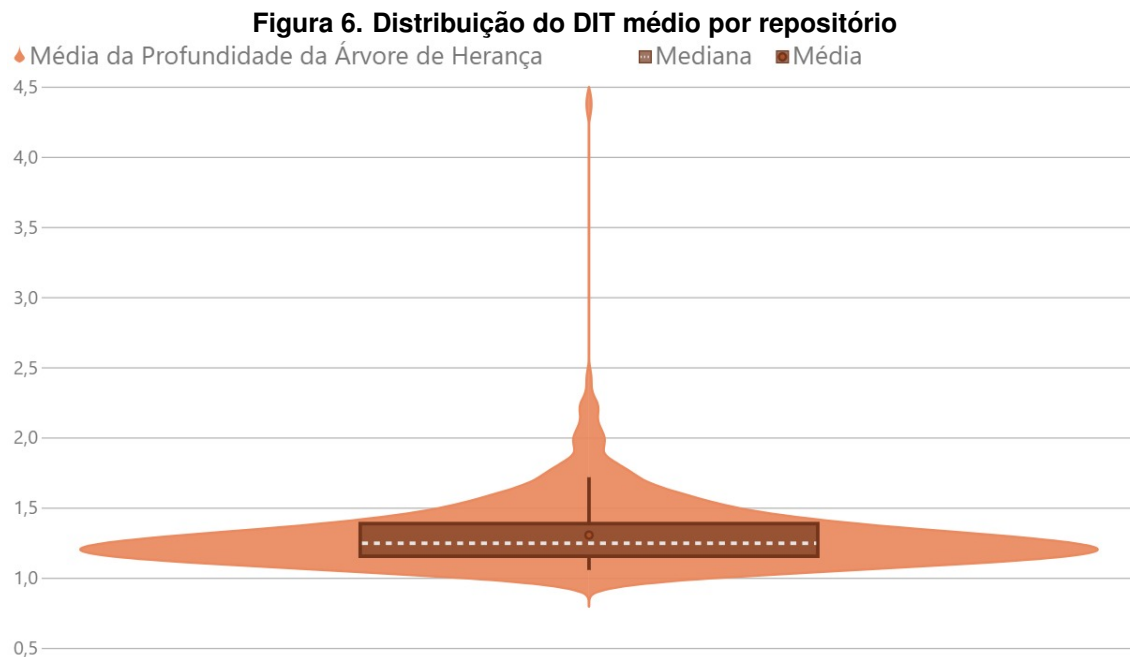
repositório, com valores variando entre 0,00 e 0,93 e média igual a 0,10. Isso indica que, na maior parte dos projetos, as classes possuem poucas ou nenhuma subclasse direta.

A Figura 6 apresenta a distribuição do DIT médio, com valores concentrados em torno de 1, e médias indicando que as classes geralmente se encontram a um ou dois níveis da raiz da hierarquia. Apenas poucos casos apresentam profundidades significativamente maiores.

A combinação de NOC baixo e DIT reduzido confirma a hipótese H3.1, indicando que a maioria dos projetos Spring Boot privilegia composição e injeção de dependências em vez de estruturas extensas de herança. Esse padrão favorece modularidade, simplicidade e testabilidade, e está alinhado ao modelo arquitetural incentivado pelo framework.



Fonte: Elaborada pelo autor



Fonte: Elaborada pelo autor

5. Discussão

Os resultados obtidos permitem identificar padrões relevantes sobre a estrutura e as práticas de desenvolvimento adotadas nos projetos Spring Boot analisados. Em relação ao acoplamento, observou-se que a maior parte dos sistemas apresenta valores moderados de CBO e RFC, o que indica que as classes tendem a depender de um conjunto limitado de outras classes. Esse comportamento é coerente com a organização em camadas e com o uso extensivo de injeção de dependências, características centrais do framework. Ainda assim, a presença de projetos com valores muito elevados sugere que certos componentes podem estar concentrando responsabilidades, o que representa um risco para evolução e manutenção.

No que diz respeito à complexidade estrutural, a associação observada entre tamanho médio dos métodos e WMC reforça a importância de práticas de decomposição e de manutenção de métodos curtos e coesos. A tendência identificada indica que, à medida que os métodos se tornam mais longos, cresce também o esforço cognitivo necessário para compreender e manter o código. Esse padrão é consistente com princípios de engenharia de software que recomendam modularidade, responsabilidade única e granularidade adequada das funções.

A análise da hierarquia de herança revelou predominância de estruturas rasas, com baixos valores de NOC e DIT. Esse achado confirma que o ecossistema Spring Boot favorece composição e uso de componentes injetados, em detrimento de hierarquias extensas de herança. Essa característica está associada a maior flexibilidade arquitetural, maior testabilidade e menor propagação de efeitos colaterais ao longo da hierarquia. Embora existam casos isolados com profundidade mais elevada, esse comportamento não representa a tendência geral da amostra.

De modo integrado, os resultados mostram que os projetos analisados seguem um padrão arquitetural alinhado às recomendações do framework, com acoplamento controlado, complexidade geralmente moderada e hierarquias simples. Entretanto, também evidenciam que existem variações importantes entre os repositórios, especialmente no que se refere à verbosidade dos métodos e ao acoplamento em projetos específicos. Esses desvios apontam para oportunidades de melhoria por meio de refatoração, revisão de responsabilidades e adoção mais consistente de boas práticas de desenvolvimento.

A combinação de métricas analisadas permite compreender não apenas a estrutura estática dos projetos, mas também possíveis implicações para manutenção futura, escalabilidade e qualidade interna do software. Esses achados reforçam a utilidade de métricas estruturais em análises exploratórias e diagnóstico arquitetural, especialmente em ecossistemas amplamente utilizados como o Spring Boot.

6. Ameaças à validade

Este estudo apresenta algumas ameaças à validade que devem ser consideradas na interpretação dos resultados. Em relação à validade de construto, as métricas extraídas dependem das definições e da implementação da ferramenta CK, que pode apresentar variações em relação a outras ferramentas de análise estática. Além disso, em projetos que utilizam injeção de dependências, parte das relações entre classes é mediada pelo contêiner, o que pode não ser totalmente capturado por métricas tradicionais de acoplamento.

Quanto à validade interna, fatores como tamanho do projeto, estilo de codificação e organização do código podem influenciar as métricas de forma indireta, introduzindo possíveis confundimentos. O estudo buscou reduzir esses efeitos por meio de métricas agregadas e análise consistente, porém não é possível eliminar completamente influências externas ao fenômeno analisado.

No que se refere à validade externa, os resultados são baseados em uma amostra de 500 repositórios coletados no GitHub, filtrados segundo critérios de tamanho, relevância e presença explícita do framework Spring Boot. Embora esses critérios garantam um nível mínimo de qualidade e maturidade, podem introduzir viés de seleção, já que projetos menores ou menos estruturados não foram incluídos. Dessa forma, os achados representam tendências observadas nesse subconjunto e podem não se generalizar integralmente para todos os projetos Java.

Também existem ameaças relacionadas à validade estatística, uma vez que a análise é baseada em métricas agregadas por repositório e não em nível de classe. A agregação pode suavizar variações internas e reduzir a sensibilidade a padrões mais específicos. Por fim, limitações procedimentais, como falhas de clonagem, repositórios indisponíveis e diferenças de ambiente, podem introduzir perdas pontuais na amostra.

Apesar dessas limitações, o estudo utilizou critérios sistemáticos de seleção, ferramenta amplamente reconhecida de extração de métricas e processo estruturado de consolidação dos dados, o que contribui para a consistência geral dos resultados e para a replicabilidade da abordagem empregada.

7. Conclusão

O estudo teve como objetivo caracterizar aspectos estruturais de projetos Spring Boot por meio de métricas orientadas a objetos extraídas com a ferramenta CK. A análise permitiu compreender como elementos como acoplamento, complexidade e uso de herança se distribuem no ecossistema considerado.

Os resultados evidenciam que os projetos seguem, em geral, padrões arquiteturais compatíveis com as recomendações do framework, mostrando níveis controlados de dependência, estruturas de herança pouco profundas e variação moderada de complexidade entre os repositórios. Embora as métricas revelem diferenças significativas entre os projetos, essas variações reforçam a importância de práticas consistentes de desenvolvimento e da adoção de estratégias que favoreçam coesão e modularidade.

O conjunto de métricas analisado demonstrou ser útil para identificar tendências estruturais e para apoiar o diagnóstico técnico de projetos baseados em Spring Boot. Como continuidade deste trabalho, recomenda-se ampliar o escopo da análise, incorporar outras métricas relacionadas ao design e explorar comparações entre diferentes frameworks e domínios, de modo a aprofundar a compreensão sobre a qualidade estrutural em sistemas modernos.

8. Pacote de Replicação

O pacote de replicação deste trabalho encontra-se disponível em:

Referências

- Anwer, S., Adbellatif, A., Alshayeb, M., and Anjum, M. S. (2017). Effect of coupling on software faults: An empirical study. In *2017 International Conference on Communication, Computing and Digital Systems (C-CODE)*, pages 211–215.
- Child, M., Rosner, P., and Counsell, S. (2019). A comparison and evaluation of variants in the coupling between objects metric. *Journal of Systems and Software*, 151:120–132.
- Gupta, V., Ganeshan, D. N., and Singhal, D. T. K. (2015). Developing software bug prediction models using various software metrics as the bug indicators. *International Journal of Advanced Computer Science and Applications*, 6(2).
- Singh, P. and Verma, S. (2012). Empirical investigation of fault prediction capability of object oriented metrics of open source software. In *2012 Ninth International Conference on Computer Science and Software Engineering (JCSSE)*, pages 323–327.
- Sun, P., Kim, D.-K., Ming, H., and Lu, L. (2022). Measuring impact of dependency injection on software maintainability. *Computers*, 11(9).
- Tahir, A., Bennin, K. E., Xiao, X., and MacDonell, S. G. (2021). Does class size matter? an in-depth assessment of the effect of class size in software defect prediction. *Empirical Software Engineering*, 26(5):106.

Apêndice J - Síntese dos gráficos das métricas estruturais

Tabela 1. Síntese dos gráficos gerados a partir das métricas estruturais

Figura	Métrica principal	Tipo de visualização	Descrição
Figura 2	<i>CBO</i> (acoplamento médio entre objetos)	<i>Boxplot</i> com distribuição por repositório	Representa a distribuição do acoplamento médio entre classes nos projetos analisados, destacando mediana, dispersão e presença de valores extremos em termos de dependências entre objetos.
Figura 3	<i>RFC</i> (resposta média para uma classe)	<i>Boxplot</i> com pontos por repositório	Apresenta a distribuição da quantidade média de métodos potencialmente acionados por classe, evidenciando a amplitude de comportamentos disponíveis e a variação entre projetos Spring Boot.
Figura 4	<i>avg loc method</i> e <i>avg wmc</i>	Gráfico de dispersão com linha de tendência	Mostra a relação entre o tamanho médio dos métodos e a complexidade média por classe, permitindo observar a tendência de aumento de complexidade estrutural em projetos com métodos mais verbosos.
Figura 5	<i>NOC</i> (número médio de filhos)	Gráfico do tipo <i>violin plot</i>	Apresenta a distribuição do número médio de subclasses diretas por classe, indicando o uso limitado de hierarquias extensas de herança na amostra de projetos Spring Boot.
Figura 6	<i>DIT</i> (profundidade média da árvore de herança)	Gráfico do tipo <i>violin plot</i>	Retrata a profundidade média da árvore de herança nos projetos, evidenciando que a maior parte das classes se encontra a um ou dois níveis da raiz, o que caracteriza hierarquias predominantemente rasas.

Elaborado pelo autor